



Alma Mater Studiorum-Università di Bologna

Scuola di Ingegneria

Fondamenti di Informatica T2

ADT Frazione: seconda parte

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



Agenda Esercitazione

- Scopo: realizzare una versione estesa di **Frazione**
- **Frazione**
 - tipo di dato astratto (ADT)
 - progettato come tipo “valore”
 - oggetto immutabile: una volta “nato” il suo valore è **costante**

Tutto adeguatamente e *costantemente* collaudato

→ test, test, test!



Frazione: puntata precedente

Funzionalità già presenti:

- **Costruire** una nuova frazione partendo da numeratore e denominatore
- **Ottenere** numeratore e denominatore della frazione
- **Verificare** se la frazione è uguale ad una data (***equals***)
- **Ottenere** una nuova frazione che rappresenti lo stesso valore in forma ***ridotta ai minimi termini***
- **Stampare** la Frazione sulla console (***toString***)



Frazione: versione estesa

Ulteriori funzionalità da realizzare:

- **Sommare** la frazione con un'altra: si ottiene una nuova frazione che rappresenta il risultato ***ridotto ai minimi termini***
- **Moltiplicare** la frazione con un'altra: si ottiene una nuova frazione che rappresenta il risultato ***ridotto ai minimi termini***
- **Sottrarre** la frazione con un'altra: si ottiene una nuova frazione che rappresenta il risultato ***ridotto ai minimi termini***
- **Dividere** la frazione con un'altra: si ottiene una nuova frazione che rappresenta il risultato ***ridotto ai minimi termini***
- **Invertire** la frazione: si ottiene una nuova frazione che rappresenta il reciproco della frazione data e che è ***già ridotta ai minimi termini***

...



Frazione: versione estesa

Ulteriori funzionalità da realizzare:

- **Confrontare** la frazione con un'altra (***compareTo***):
il risultato dev'essere un intero e precisamente
 - 0, se le due frazioni sono *uguali*
 - 1, se la Frazione su cui è invocato il metodo è *maggiore* della Frazione ricevuta in ingresso
 - -1, se la Frazione su cui è invocato il metodo è *minore* della Frazione ricevuta in ingresso
- **Recuperare il valore della frazione come double** (***getDouble***):
il risultato dev'essere un *valore di tipo double* che rappresenti il valore della funzione

Analisi dei requisiti: sum

Metodo classico, sfruttando *mcm*:

$$8/5 + 6/15 = (24 + 6) / 15 = 2/1$$

```
public Frazione sumWithMcm (Frazione f) {  
    int mcm = MyMath.mcm(f.den, this.den);  
    ...  
}
```

Usare il metodo **mcm**
dalla libreria **MyMath**

Metodo alternativo

$$8/5 + 6/15 = (8*15 + 6*5) / (15*5) = (120+30) / 75 = 150/75 = 2/1$$

```
public Frazione sum(Frazione f) {  
    int n = num * f.den + den * f.num;  
    int d = den * f.den;  
    Frazione fSum = new Frazione(n, d);  
    return fSum.minTerm();  
}
```

Analogamente si procede
per le altre operazioni

Analisi dei requisiti : il modello

Frazione
- den: int - num: int
+ compareTo(f: Frazione): int + div(f: Frazione): Frazione + equals(f: Frazione): boolean + Frazione(num: int, den: int) + Frazione(num: int) + getDen(): int + getDouble(): double + getNum(): int + minTerm(): Frazione + mul(f: Frazione): Frazione + reciprocal(): Frazione + sub(f: Frazione): Frazione + sum(f: Frazione): Frazione + sumWithMcm(f: Frazione): Frazione + toString(): String



E ora... al lavoro!

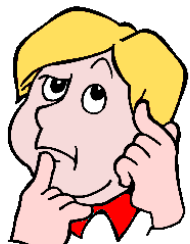
Scopo: completare l'ADT con i metodi della versione estesa

- Metodo `sum` e `sumWithMcm` per sommare la frazione con un'altra,
- Metodo `sub` per sottrarre la frazione con un'altra
- Metodo `mul` per moltiplicare la frazione con un'altra
- Metodo `div` per dividere
- Metodo `reciprocal` per invertire la frazione
- Metodo `compareTo` per confrontare la frazione con un'altra
- Metodo `getDouble` per restituire il valore reale equivalente

NB: collaudo sempre continuo e sistematico!

Trattamento degli errori

- L'attuale *contratto di utilizzo* prevede che colui che crea la frazione sia *responsabile del risultato*: creare una frazione con zero al denominatore è possibile... ma prima o poi scoppia tutto!
- Il bug è evidentemente a MONTE della costruzione, ma se ne subiscono le conseguenze a VALLE, quando si usa l'oggetto
- IDEALMENTE, bisognerebbe poter *interrompere l'operazione* e *notificare al chiamante* il problema, ma (per ora!) non sappiamo farlo
- In alternativa, ci vorrebbe qualcuno che almeno verificasse i parametri PRIMA della costruzione....



Idee?



StartKit + Struttura Progetto

- Nello StartKit troverete:
 - Frazione nella versione base
 - FrazioneTest che contiene i test
 - MainFrazione che contiene un main di prova
 - MyMath che contiene la libreria matematica

